

FastAPI Backend Security Audit — Ignition

Repository	Ignition backend (github.com/NlogN-Solution/Ignition-Backend)
Branch	main
Commit SHA	1e58579de9f53f9bce5494e1528e3aae74b0e057
Audit date	2026-09-25
Auditor	Claude Code (claude-opus-5-5)
Specification	FastAPI_Backend_Security_Audit_Claude_Code.pdf (hybrid review, OWASP API Security Top 10 2023)
Scope	Whole backend repository: app/ (30.8k LOC, 339 HTTP endpoints across 29 routers), configuration, Dockerfile, docker-compose.yml, render.yaml, migrations (inventory only), scripts/ (import tooling), tests/.

1. Executive summary

Ignition's backend is well defended at the level most APIs get wrong. Every one of the 339 routes declares an authentication dependency, and a test fails the build if one does not. Public self-registration cannot set a role. JWT handling is correct, refresh tokens are rotated and stored server-side, uploaded file names are discarded, message HTML is sanitised on input, and the student portal reads through a repository that always filters by the calling student.

The weaknesses are in the next layer down: checks on which object a request may touch, and on business flows. The audit confirmed 12 vulnerabilities (no Critical or High; 4 Medium, 8 Low) and 6 informational hardening items. All confirmed vulnerabilities except the two that depend on the deployment environment were reproduced with local, non-destructive regression tests against the `ignition_test` database.

The highest-priority problems:

- FAPI-SEC-001: the simulated checkout is on by default and nothing turns it off in production, so any self-registered student can unlock the paid portal for free.
- FAPI-SEC-002: a checklist item accepts any document id. Students can self-verify requirements with an unrelated approved file, or link another student's document.
- FAPI-SEC-005: the counsellor 'own work only' privacy rule, enforced on `/applications`, `/leads` and the legacy `/messages`, is missing from the new `/communication` router (including internal staff notes) and from the application workflow/checklist routes.
- FAPI-SEC-010 (needs environment verification): rate limits key on the Render proxy's IP, which turns per-client login limits into one global bucket that an anonymous caller can exhaust.

Overall risk posture

Moderate. No finding gives an unauthenticated attacker access to personal data or privileged functions. The confirmed issues need an account (free for students) or a staff role, and several need a victim's random UUID. The business-flow bypass (FAPI-SEC-001) and the internal-privacy gap (FAPI-SEC-005) are the ones with direct real-world consequences. No numeric score is given, by design.

2. Scope and exclusions

In scope: Whole backend repository: `app/` (30.8k LOC, 339 HTTP endpoints across 29 routers), configuration, `Dockerfile`, `docker-compose.yml`, `render.yaml`, migrations (inventory only), `scripts/` (import tooling), `tests/`.

Excluded: Frontends (Ignition-AdminFrontend, Ignition-StudentDashboard, Ignition-Landing), the landing site's `/api/revalidate` and `/api/preview` handlers, Cloudinary/Neon/Render account configuration, production runtime. No production or third-party system was contacted; all verification ran against the local `ignition_test` database inside a rolled-back transaction.

3. Repository and architecture overview

- Stack: Python 3.12, FastAPI 0.141.1 / Starlette 1.3.1, Uvicorn 0.52, Pydantic 2.13, SQLAlchemy 2.0 (asyncpg), Alembic, PyJWT 2.13 (HS256), bcrypt 5.0, slowapi (Redis-backed), Cloudinary for media, httpx for the landing revalidation webhook.
- Entry point `app/main.py`: CORS (explicit origins; LAN regex only in development), a request-id middleware, a generic 500 handler (no stack traces to clients), and OpenAPI/docs disabled when `ENVIRONMENT=production`.
- 29 routers mounted under `/api/v1` by `app/api/router.py`; no WebSockets, SSE, static mounts or background workers (events run in-process after commit; the docker-compose Celery worker points to a module that does not exist).
- Data: single-tenant PostgreSQL (Neon in production), Redis (rate limits, dashboard cache, public response cache).

- Deployment: Docker (non-root uid 1001, slim image), Render Blueprint (render.yaml) running `alembic upgrade head` before each deploy.
- Trust boundaries: anonymous (public catalogue, eligibility, apply-intent, register/login); student (self-registered); staff roles (viewer, counsellor, frontdesk, staff, finance, marketing, support, admissions, manager, admin, super_admin); external services (Cloudinary, landing site webhook).

4. Authentication and authorization architecture

- Bearer JWT (HTTPBearer). Access tokens last 15 minutes with type=access; refresh tokens last 30 days, carry a jti, are stored as a SHA-256 hash in user_sessions, rotate on every use and are revoked on logout and password change. Algorithms are pinned to settings.JWT_ALGORITHM, so alg=none is rejected (verified). A refresh token used as a bearer token is rejected (verified).
- Production refuses to boot with a missing, weak (<32 chars) or placeholder JWT secret, or without Cloudinary keys. Development generates a per-process secret.
- Passwords: bcrypt (cost 12) over base64(SHA-256(password)), which avoids 72-byte truncation. Timing is equalised for unknown accounts. Lockout after 5 failures for 15 minutes (see FAPI-SEC-011).
- Authorization dependencies: get_current_user (active, not deleted, checked per request), require_role (super_admin always passes), require_owner, require_staff, get_current_student, require_public. tests/test_endpoint_authorization.py fails the build if a route has no marker or if the public set changes.
- Object level: StudentScopedRepository for the student portal; inline owner checks on documents, appointments, payments, notifications, leave, payroll and attendance; can_manage_target rank rule for user administration; counsellor own-work scoping (api/scoping.py, services/message_scope.py) on leads, applications and legacy messages.
- No cookies, so CSRF does not apply. There are no password-reset or e-mail-verification flows yet: staff reset passwords administratively and temporary passwords are returned once.

5. Attack surface and endpoint inventory summary

Metric	Count
HTTP endpoints discovered (method + path)	339
Route decorators in app/routes (cross-check)	339
Unauthenticated endpoints	21
Authenticated, no role restriction (object checks inline)	59
Student-role endpoints	95
Endpoints mapped to at least one finding	55
Endpoints marked Reviewed in endpoint-security-matrix.csv	339

Unauthenticated surface:

Method	Path
POST	/api/v1/auth/login
POST	/api/v1/auth/refresh

Method	Path
POST	/api/v1/auth/register
GET	/api/v1/health
GET	/api/v1/health/ready
POST	/api/v1/public/apply-intents
GET	/api/v1/public/apply-intents/{intent_id}
GET	/api/v1/public/content
GET	/api/v1/public/content/{key}
GET	/api/v1/public/course-profiles
GET	/api/v1/public/course-profiles/{slug}
GET	/api/v1/public/courses
GET	/api/v1/public/courses/facets
GET	/api/v1/public/courses/{slug}
POST	/api/v1/public/eligibility
GET	/api/v1/public/posts
GET	/api/v1/public/posts/{slug}
GET	/api/v1/public/scholarships
GET	/api/v1/public/taxonomies
GET	/api/v1/public/universities
GET	/api/v1/public/universities/{slug}

The full per-route matrix (authentication, role dependency, object ids, object-level control, body, response model, review status, findings) is in endpoint-security-matrix.csv.

6. Methodology and tools actually executed

- Phase 0, inventory: repository layout, configuration, deployment files. The route inventory was built by introspecting the live FastAPI app with the project's own traversal (FastAPI 0.141 nests included routers) and cross-checked against the 339 decorators in app/routes.
- Phase 1, deterministic: pytest (existing suite), ruff, mypy. Bandit, pip-audit, Semgrep and a secret scanner are not installed, and per the specification were not installed silently (see section 10).
- Phase 2, manual semantic review of every router and the services behind each object-level decision, against OWASP API Top 10 2023 and the FastAPI-specific checklist.
- Phase 3, source-to-sink tracing for each candidate: request input, dependency, ownership check, service, then SQL/storage sink.

- Phase 4, safe verification: 30 pytest cases in security-audit/tests/security/ using the project's rolled-back ignition_test harness and synthetic users only. 16 reproduce findings; 14 are positive controls that pass.
- Phase 5, triage: severity by likelihood and impact, grouped by root cause, false positives recorded.

7. Findings summary

Severity	Count
Critical	0
High	0
Medium	4
Low	8
Informational	6

ID	Severity	Confidence	Title	Affected area	Status
FAPI-SEC-001	Medium	High	Simulated portal checkout is enabled by default in production, so any student can unlock the paid portal without paying	POST /api/v1/student/me/access/checkout	Confirmed
FAPI-SEC-002	Medium	High	Checklist items accept any document_id: students can self-verify requirements and link other students' documents	PATCH /api/v1/applications/{application_id}/checklist/{item_id}	Confirmed
FAPI-SEC-005	Medium	High	Counsellor own-work scoping is not enforced on the /communication router or the application workflow/checklist routes	GET /api/v1/communication/threads	Confirmed
FAPI-SEC-010	Medium	Medium	Rate limits and audit IPs are keyed on the reverse proxy's address, not the client's	POST /api/v1/auth/login (10/min)	Needs Environment Verification

ID	Severity	Confidence	Title	Affected area	Status
FAPI-SEC-003	Low	High	Workflow step ids are not tied to the application in the URL (cross-application read and write)	GET /api/v1/applications/{application_id}/workflow/steps/{step_id}/activities	Confirmed
FAPI-SEC-004	Low	High	Client-supplied application_id and intake_id are not ownership-checked when documents, threads and applications are created	POST /api/v1/documents/upload (form field application_id)	Confirmed
FAPI-SEC-006	Low	High	Leave-request attachments (e.g. medical notes) are uploaded as public Cloudinary assets	POST /api/v1/leave-requests	Confirmed

ID	Severity	Confidence	Title	Affected area	Status
FAPI-SEC-007	Low	High	Pagination parameters are unbounded and unvalidated on about 45 list endpoints	Every authenticated list route declaring page: int = 1, limit: int = N without Query(generator): users, documents, documents/folders, applications, leads, appointments, payments, tasks, notifications, content-*, catalogue, academic, attendance, leave, payroll, activity-logs, workflow, student/me/* lists (see endpoint-security-matrix.csv)	Confirmed

ID	Severity	Confidence	Title	Affected area	Status
FAPI-SEC-008	Low	High	Unpublished CMS and catalogue drafts are readable by any authenticated account, including self-registered students	GET /api/v1/content-pages/{id}]	Confirmed
FAPI-SEC-009	Low	High	Account e-mail handling: case-sensitive uniqueness, unverified self-service e-mail change, duplicate address returns 500	POST /api/v1/auth/register	Confirmed
FAPI-SEC-011	Low	High	Account lockout enables targeted lockout of any account and confirms which e-mails are registered	POST /api/v1/auth/login	Confirmed
FAPI-SEC-012	Low	Medium	Uploads are read fully into memory before the size check, with no request-body cap	POST /api/v1/documents/upload	Needs Environment Verification
FAPI-SEC-013	Informational	High	Signed Cloudinary delivery URLs for private documents never expire	GET /api/v1/documents/{document_id}/link download	Informational
FAPI-SEC-014	Informational	High	Counsellors can read and edit any student's profile and documents regardless of assignment	/api/v1/users/{user_id}/student-profile education experience research shortlist	Informational
FAPI-SEC-015	Informational	High	Managers and admins can approve their own leave requests	POST /api/v1/leave-requests/{request_id}/approve	Informational

ID	Severity	Confidence	Title	Affected area	Status
FAPI-SEC-016	Informational	High	Access tokens survive logout and password change; refresh rotation is not atomic	POST /api/v1/auth/logout	Informational
FAPI-SEC-017	Informational	Medium	The developer .env holds hosted Neon database and Cloudinary credentials under ENVIRONMENT=development	n/a (developer workstation)	Needs Environment Verification
FAPI-SEC-018	Informational	Medium	Anonymous eligibility submissions attach to existing leads matched by e-mail or phone	POST /api/v1/public/eligibility	Informational

FAPI-SEC-001 to 012 are vulnerabilities. FAPI-SEC-013 to 018 are hardening recommendations or environment-dependent observations, kept separate as the specification requires.

8. Detailed findings

FAPI-SEC-001 — Simulated portal checkout is enabled by default in production, so any student can unlock the paid portal without paying

Severity **Medium**

Confidence High

Status Confirmed

CWE CWE-1188: Insecure Default Initialization of Resource (with CWE-840 Business Logic Errors)

OWASP API API6:2023 Unrestricted Access to Sensitive Business Flows; API8:2023 Security Misconfiguration

Affected endpoints

- POST /api/v1/student/me/access/checkout
- GET /api/v1/documents/{document_id}/link
- GET /api/v1/documents/{document_id}/download

Affected code

- app/core/config.py:40
- app/core/config.py:141-158
- app/routes/student.py:2047-2089
- app/routes/document.py:76-103
- render.yaml:48-79

Evidence / code path. Settings.SIMULATED_PAYMENTS defaults to True (config.py:40). The production validator (_validate_secrets, config.py:141) checks the JWT secret and Cloudinary keys but never SIMULATED_PAYMENTS, and render.yaml does not set it, so a production deploy from the Blueprint runs with True. checkout_my_portal_access (student.py:2052) only refuses when the flag is False; otherwise it writes a COMPLETED PORTAL_ACCESS Payment (student.py:2072). PortalAccessService.has_access returns True

for any COMPLETED PORTAL_ACCESS payment, which is exactly what _assert_entitled (document.py:99) checks before releasing offer and CAS letters.

Preconditions. Any self-registered student account (registration is public). Production deployed without SIMULATED_PAYMENTS=false in the environment.

Attack scenario. A student registers, calls POST /student/me/access/checkout with {"payment_method":"esewa"}, receives has_access=true, and downloads their offer/CAS letters without paying the access fee.

Impact. Complete bypass of the portal access fee, which is the product's payroll, and loss of that revenue. The simulated payment rows are marked SIMULATED in reference and remarks, so finance can spot them afterwards, but access is already granted.

Safe reproduction / regression test. security-audit/tests/security/test_audit_findings.py::test_fapi_sec_001_production_refuses_simulated_payments and ::test_fapi_sec_001_checkout_unlocks_gated_documents_without_payment (both FAIL on the audited commit: production Settings boot with SIMULATED_PAYMENTS=True; checkout returns 200).

Root cause. A demo-only switch fails open: its safe value has to be set explicitly, and nothing forces that in production.

Remediation. Make production refuse to boot while SIMULATED_PAYMENTS is true (or default it to False and enable it only in development/test), and set SIMULATED_PAYMENTS=false in render.yaml. Consider also refusing checkout when ENVIRONMENT=production regardless of the flag until a real gateway is integrated.

Minimal patch (illustrative, not applied)

```
--- a/app/core/config.py
+++ b/app/core/config.py
@@ def _validate_secrets(self) -> Settings:
     if self.ENVIRONMENT == "production":
+         if self.SIMULATED_PAYMENTS:
+             raise ValueError("SIMULATED_PAYMENTS must be false in production (no gateway is
integrated).")
     if not self.JWT_SECRET_KEY:
--- a/render.yaml
+++ b/render.yaml
- key: ENVIRONMENT
  value: production
+ - key: SIMULATED_PAYMENTS
+   value: "false"
```

Verification after fix. Not tested (fixes not applied, per audit rules)

FAPI-SEC-002 — Checklist items accept any document_id: students can self-verify requirements and link other students' documents

Severity **Medium**

Confidence High

Status Confirmed

CWE CWE-639: Authorization Bypass Through User-Controlled Key; CWE-863: Incorrect Authorization

OWASP API API1:2023 Broken Object Level Authorization; API6:2023 Unrestricted Access to Sensitive Business Flows

Affected endpoints

- PATCH /api/v1/applications/{application_id}/checklist/{item_id}
- GET /api/v1/student/me/applications/{application_id}/documents (disclosure sink)

Affected code

- app/routes/workflow.py:485
- app/routes/workflow.py:493-511
- app/services/workflow_service.py:630-664
- app/routes/student.py:866-905

Evidence / code path. A student may PATCH document_id (_STUDENT_EDITABLE_CHECKLIST_FIELDS, workflow.py:485). ChecklistService.update_item loads the document by id (workflow_service.py:650) but checks neither its owner nor its type. If the document is already APPROVED, the item is set to VERIFIED (workflow_service.py:655), and link_to_application then files the document on the application (workflow_service.py:664). The student's application-documents list selects linked documents by application_id only (student.py:893), not by Document.student_id, so the linked foreign document's metadata is returned to the caller.

Preconditions. Authenticated student with an application that has a checklist item. Self-verification needs nothing more than any of their own APPROVED documents. Cross-student linking also needs another student's document UUID (random UUIDv4, so hard to obtain).

Attack scenario. (a) A student whose 'photo' was approved links it to the 'passport' checklist item; the item is VERIFIED immediately and no reviewer ever sees a passport. (b) With a leaked document id, a student links another applicant's approved passport to their own application; it shows as verified evidence and its title, filename, remarks and status appear in the attacker's application view.

Impact. The document-review workflow can be bypassed: staff see VERIFIED requirements that were never reviewed for that purpose. It also allows cross-student linking and disclosure of another student's document metadata (the file bytes stay protected by the owner check on /documents/{id}/download).

Safe reproduction / regression test. test_fapi_sec_002_student_cannot_link_another_students_document (FAIL: status=200, item status=verified) and test_fapi_sec_002_approved_document_of_other_type_does_not_self_verify (FAIL: item VERIFIED).

Root cause. The child resource (document) named in the request body is trusted without checking that it belongs to the same student as the parent application, and without checking that its type matches what the item requested.

Remediation. In update_application_checklist_item, resolve the document and require document.student_id == application.student_id (404 otherwise). Require document.document_type to match item.document_type when the item names one. Only auto-VERIFY when the approval came from staff for this same requirement; otherwise land SUBMITTED. Also restrict the student application-documents list to Document.student_id == application.student_id as defence in depth.

Minimal patch (illustrative, not applied)

```
--- a/app/routes/workflow.py
+++ b/app/routes/workflow.py
@@ async def update_application_checklist_item(
-     await _assert_application_access(application_id, user, application_service)
+     application = await _assert_application_access(application_id, user, application_service)
@@
     data = payload.model_dump(exclude_unset=True)
+     if data.get("document_id") is not None:
+         document = await DocumentService(application_service.session).get_document(data["document_id"])
+         if document is None or document.student_id != application.student_id:
+             raise NotFoundException("Document not found")
+         if item.document_type is not None and document.document_type is not item.document_type:
+             raise BadRequestException("That document is not the type this item asks for")
```

Verification after fix. Not tested

FAPI-SEC-003 — Workflow step ids are not tied to the application in the URL (cross-application read and write)

Severity	Low
Confidence	High
Status	Confirmed
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
OWASP API	API1:2023 Broken Object Level Authorization

Affected endpoints

- GET /api/v1/applications/{application_id}/workflow/steps/{step_id}/activities
- PATCH /api/v1/applications/{application_id}/workflow/steps/{step_id}
- POST /api/v1/applications/{application_id}/workflow/steps/{step_id}/activities

Affected code

- app/routes/workflow.py:384-397
- app/routes/workflow.py:404-412
- app/routes/workflow.py:420-432
- app/services/workflow_service.py:372-376
- app/services/workflow_service.py:491-497

Evidence / code path. Each handler authorises the caller against application_id (`_assert_application_access`) and then acts on step_id with `get_step(step_id)/list_activities(step_id)`, which filter on the step id only. Nothing checks that the step's `ApplicationWorkflow.application_id` equals the application_id in the path.

Preconditions. Student: owns any application and knows another application's step UUID. Staff (admin/counsellor): can already reach any application through these routes (see FAPI-SEC-005), so the extra exposure is mostly for students.

Attack scenario. A student calls GET /applications/{own_app}/workflow/steps/{victim_step}/activities and reads staff comments and status history on another student's workflow.

Impact. Cross-student disclosure of workflow activity (staff comments). Counsellors can also change steps of applications outside their own-work scope. Likelihood is limited because step ids are random UUIDs that are only shown to people who can already see that workflow.

Safe reproduction / regression test. `test_fapi_sec_003_student_cannot_read_other_applications_step_activities` (FAIL: returned `[None, 'SYNTHETIC-PRIVATE-NOTE']`).

Root cause. The parent/child relationship is checked for the parent only; the child lookup is not scoped to the parent.

Remediation. Add `ApplicationWorkflowService.get_step_for_application(step_id, application_id)`, which joins `ApplicationWorkflow` and filters on `application_id`, and use it in all three step routes. Return 404 on mismatch.

Minimal patch (illustrative, not applied)

```
+ async def get_step_for_application(self, step_id: UUID, application_id: UUID) -> ApplicationWorkflowStep
+ | None:
+     return await self.session.scalar(
+         select(ApplicationWorkflowStep)
+         .join(ApplicationWorkflow, ApplicationWorkflow.id ==
ApplicationWorkflowStep.application_workflow_id)
+         .where(ApplicationWorkflowStep.id == step_id, ApplicationWorkflow.application_id ==
application_id)
+     )
# routes/workflow.py: replace get_step(step_id) with get_step_for_application(step_id, application_id) and 404
```

```
on None,  
# including before list_activities(step_id).
```

Verification after fix. Not tested

FAPI-SEC-004 — Client-supplied application_id and intake_id are not ownership-checked when documents, threads and applications are created

Severity Low
Confidence High
Status Confirmed
CWE CWE-639: Authorization Bypass Through User-Controlled Key
OWASP API API1:2023 Broken Object Level Authorization

Affected endpoints

- POST /api/v1/documents/upload (form field application_id)
- POST /api/v1/student/me/threads (application_id)
- POST /api/v1/student/me/applications (intake_id)

Affected code

- app/routes/document.py:176
- app/routes/document.py:221
- app/services/document_service.py:65-71
- app/routes/student.py:738
- app/routes/student.py:478

Evidence / code path. upload_document forces student_id to the caller for students but passes the form's application_id straight to DocumentService.create_document, which inserts an ApplicationDocument link (document_service.py:70) with no ownership check. create_my_thread passes payload.application_id into create_thread unchecked. create_my_application stores payload.intake_id without checking that it belongs to payload.program_id. A nonexistent id raises an IntegrityError, which returns 500.

Preconditions. Authenticated student who knows another student's application UUID.

Attack scenario. A student uploads a file with application_id set to a victim's application. The file then appears in the victim's 'documents filed against this application' view and on the staff side of that application. A thread can be filed against the victim's application in the same way.

Impact. Integrity: unsolicited content (possibly malicious files) planted on another applicant's file, where staff and the victim will open it. There is no read access to the victim's data. Likelihood is low because application ids are random UUIDs.

Safe reproduction / regression test. test_fapi_sec_004_upload_cannot_attach_to_foreign_application (FAIL: status=200) and test_fapi_sec_004_student_thread_cannot_attach_to_foreign_application (FAIL: status=201).

Root cause. Body/form foreign keys to parent resources are trusted; only the path parameters go through ownership checks.

Remediation. When application_id is supplied, load the application and require application.student_id == student_id (for staff uploads too), returning 404 otherwise. For student threads use repo.get_or_404(Application, payload.application_id). Validate that intake_id belongs to program_id.

Minimal patch (illustrative, not applied)

```
--- a/app/routes/document.py  
+++ b/app/routes/document.py  
@@ async def upload_document(
```

```

    if user.role is UserRole.STUDENT:
        student_id = user.id
+   if application_id is not None:
+       application = await ApplicationService(service.session).get_application(application_id)
+       if application is None or application.student_id != student_id:
+           raise NotFoundException("Application not found")
--- a/app/routes/student.py (create_my_thread)
+   if payload.application_id is not None:
+       await StudentScopedRepository(service.session, student).get_or_404(Application,
payload.application_id)

```

Verification after fix. Not tested

FAPI-SEC-005 — Counsellor own-work scoping is not enforced on the /communication router or the application workflow/checklist routes

Severity **Medium**

Confidence High

Status Confirmed

CWE CWE-285: Improper Authorization

OWASP API API1:2023 Broken Object Level Authorization

Affected endpoints

- GET /api/v1/communication/threads
- GET /api/v1/communication/students/{student_id}/threads
- GET /api/v1/communication/leads/{lead_id}/threads
- GET /api/v1/communication/applications/{application_id}/threads
- GET /api/v1/communication/threads/{thread_id}
- POST /api/v1/communication/threads/{thread_id}/messages
- GET /api/v1/communication/attachments/{attachment_id}/link | download
- GET | POST | PATCH /api/v1/applications/{application_id}/workflow[/...]
- GET | POST | PATCH /api/v1/applications/{application_id}/checklist[/...]

Affected code

- app/routes/communication.py:129-147
- app/routes/communication.py:174-192
- app/routes/communication.py:233-250
- app/services/communication_service.py:160-202
- app/routes/workflow.py:63-83

Evidence / code path. The codebase defines a privacy rule: a counsellor sees their own records plus unclaimed ones (api/scoping.py:45, services/message_scope.py:41). It is enforced on /applications/{id} (application.py:53), /leads and the legacy /messages router. The /communication router, which replaces /messages, never applies it: staff_inbox takes no user and returns every thread, list_student_threads defaults include_internal=True, and get_thread only checks the role (communication.py:247). _assert_application_access returns any application for any APPLICATION_STAFF_ROLES member (workflow.py:79) without may_see_record, although GET /applications/{id} returns 404 for the same caller.

Preconditions. Authenticated counsellor (or admissions/manager) account.

Attack scenario. Counsellor A opens /communication/threads or /communication/students/{id}/threads for a student counselled by counsellor B and reads the whole correspondence, including INTERNAL staff notes. The same counsellor reads and edits the workflow and checklist of B's applications, even though /applications/{id} hides them.

Impact. Confidentiality breach of student correspondence, which the code itself calls 'the clearest privacy failure in the console'. Internal notes are exposed across the counsellor team, and the scoping on /applications is undone by the sub-resources.

Safe reproduction / regression test. test_fapi_sec_005_other_counsellors_student_correspondence (FAIL: 2 threads incl. internal; inbox shows 2), test_fapi_sec_005_checklist_of_other_counsellors_application (FAIL: 200), and positive control test_fapi_sec_005_control_application_is_hidden_from_other_counsellor (PASS: 404).

Root cause. Scoping is applied per router rather than centrally, and the new router and the sub-resource helper were written without it.

Remediation. Add a thread-level visibility condition (own student, own lead, own application, already replied, or unclaimed) to staff_inbox, threads_for_student/lead/application and the by-id thread and attachment checks for non-admin roles. Return 404 when it fails. In _assert_application_access, call may_see_record(user, application.counsellor_id) for staff and return 404 when it fails. Add scoping tests mirroring tests/test_scoping.py for these routes.

Minimal patch (illustrative, not applied)

```
--- a/app/routes/workflow.py
+++ b/app/routes/workflow.py
@@ async def _assert_application_access(
-     if user.role in APPLICATION_STAFF_ROLES:
-         return application
+     if user.role in APPLICATION_STAFF_ROLES:
+         if not may_see_record(user, application.counsellor_id):
+             raise NotFoundException("Application not found")
+         return application
# communication: pass `user` into staff_inbox/threads_for_* and AND a visibility clause
# derived from message_scope.visible_students_condition (keyed on MessageThread.student_id /
lead.assigned_to).
```

Verification after fix. Not tested

FAPI-SEC-006 — Leave-request attachments (e.g. medical notes) are uploaded as public Cloudinary assets

Severity **Low**

Confidence High

Status Confirmed

CWE CWE-552: Files or Directories Accessible to External Parties

OWASP API API3:2023 Broken Object Property Level Authorization (data exposure); API8:2023 Security Misconfiguration

Affected endpoints

- POST /api/v1/leave-requests
- GET /api/v1/leave-requests[/{request_id}] (returns attachment_url)

Affected code

- app/routes/leave.py:132
- app/core/uploads.py:99
- app/core/uploads.py:142
- app/core/uploads.py:59-63

Evidence / code path. store_upload defaults to private=False, which uploads as type='upload' (public, CDN-served, no signature). create_leave_request calls it without private=True (leave.py:132) and stores the returned public secure_url. The uploads module

states that CONTENT_FOLDER is 'the one folder uploaded with private=False'; leave attachments contradict that design.

Preconditions. Anyone who obtains the URL: browser history, logs, referrers, a forwarded link, or any manager/admin viewing the request.

Attack scenario. A leave request's attachment URL is copied into a chat or ticket. Anyone holding it can fetch the employee's medical certificate indefinitely, with no authentication and no expiry.

Impact. Exposure of employee health data. Likelihood is low because the object name is a random UUID, but there is no access control at all once the URL is known.

Safe reproduction / regression test. test_fapi_sec_006_leave_attachment_is_private (FAIL: attachment_url returned, which store_upload only does for public uploads).

Root cause. A default-public storage helper was used for a sensitive document class.

Remediation. Upload leave attachments with private=True into LEAVE_ATTACHMENT_FOLDER, store stored_file_name instead of a URL, and serve them through an authenticated route that checks self-or-MANAGE_ROLES before minting a signed URL. Migrate existing assets to the authenticated delivery type. Consider making store_upload's private parameter keyword-required with no default.

Minimal patch (illustrative, not applied)

```
--- a/app/routes/leave.py
+++ b/app/routes/leave.py
-     stored = await store_upload(file, DOCUMENT_EXTENSIONS, folder=LEAVE_ATTACHMENT_FOLDER)
-     attachment_url = stored.url
+     stored = await store_upload(file, DOCUMENT_EXTENSIONS, folder=LEAVE_ATTACHMENT_FOLDER, private=True)
+     attachment_url = f"/api/v1/leave-requests/{{id}}/attachment" # authenticated, owner-or-manager
```

Verification after fix. Not tested

FAPI-SEC-007 — Pagination parameters are unbounded and unvalidated on about 45 list endpoints

Severity **Low**

Confidence High

Status Confirmed

CWE CWE-770: Allocation of Resources Without Limits or Throttling; CWE-20: Improper Input Validation

OWASP API API4:2023 Unrestricted Resource Consumption

Affected endpoints

- Every authenticated list route declaring `page: int = 1, limit: int = N` without `Query(ge=1, le=N)`: users, documents, documents/folders, applications, leads, appointments, payments, tasks, notifications, content-*, catalogue, academic, attendance, leave, payroll, activity-logs, workflow, student/me/* lists (see endpoint-security-matrix.csv)

Affected code

- app/api/student.py:136
- app/services/document_service.py:61
- app/services/document_service.py:266
- app/routes/public.py:88 (the correct pattern, used only on /public)

Evidence / code path. Only /public/* and /communication/threads constrain limit with `Query(ge=1, le=100)`. Everywhere else limit and page are plain ints passed straight into `.limit().offset((page-1)*limit)`. `page=0` produces a negative OFFSET: `asynccpg` raises

InvalidRowCountInResultOffsetClauseError, which becomes a 500. limit=1000000 is accepted.

Preconditions. Any authenticated user (students included for the /student/me/* and shared lists).

Attack scenario. A script repeatedly calls list endpoints with limit=1000000 (for example /documents/folders, /content-pages or the /student/me/* lists), forcing large serialisations and database scans; page=0 or negative limits generate 500s and error-log noise.

Impact. Availability and cost: memory and CPU spikes on a small Render instance, plus log flooding.

Safe reproduction / regression test. test_fapi_sec_007_limit_is_clamped (FAIL: 200) and test_fapi_sec_007_page_zero_is_a_client_error (FAIL: 500, negative OFFSET).

Root cause. The clamping convention was applied to the public router only, not project-wide.

Remediation. Move PageParam/LimitParam (ge=1, le=100) from routes/public.py into a shared module and use them on every list route. Also clamp inside StudentScopedRepository.list and the services as defence in depth.

Minimal patch (illustrative, not applied)

```
# app/api/pagination.py
+PageParam = Annotated[int, Query(ge=1)]
+LimitParam = Annotated[int, Query(ge=1, le=100)]
# every list route:
-   page: int = 1,
-   limit: int = 20,
+   page: PageParam = 1,
+   limit: LimitParam = 20,
```

Verification after fix. Not tested

FAPI-SEC-008 — Unpublished CMS and catalogue drafts are readable by any authenticated account, including self-registered students

Severity

Low

Confidence High

Status Confirmed

CWE CWE-285: Improper Authorization

OWASP API API5:2023 Broken Function Level Authorization

Affected endpoints

- GET /api/v1/content-pages/{id}
- GET /api/v1/content-blocks/{id}
- GET /api/v1/media-assets/{id}
- GET /api/v1/blog-posts/{id}
- GET /api/v1/country-guides/{id}
- GET /api/v1/university-routes/{id}
- GET /api/v1/course-profiles/{id}
- GET /api/v1/scholarships/{id}

Affected code

- app/routes/content.py:75-99 (and :217, :227, :280, :290, :378, :388, :441, :451)
- app/routes/catalogue.py:68, :80, :134, :146, :200, :212

Evidence / code path. The staff CMS and catalogue read routes depend only on `get_current_user` and accept `is_published=None/False`. Because `/auth/register` is public, 'authenticated' effectively means 'anyone'. Writes are correctly limited to `admin/marketing`.

Preconditions. Any account (free self-registration).

Attack scenario. A competitor registers a student account and lists `/content-pages?is_published=false` and `/scholarships` to read embargoed pages, draft pricing and unpublished scholarships before launch.

Impact. Early disclosure of unpublished marketing and catalogue content. No personal data is involved.

Safe reproduction / regression test. `test_fapi_sec_008_student_cannot_read_unpublished_content` (FAIL: student read an unpublished draft, 200).

Root cause. The staff console's read routes use the weakest authentication dependency.

Remediation. Guard these reads with `require_staff` (or the same `admin/marketing` role set as the writes). Students already have published-only equivalents under `/public` and `/student/catalog`.

Minimal patch (illustrative, not applied)

```
-   _: object = Depends(get_current_user),
+   _: object = Depends(require_staff),
```

Verification after fix. Not tested

FAPI-SEC-009 — Account e-mail handling: case-sensitive uniqueness, unverified self-service e-mail change, duplicate address returns 500

Severity **Low**

Confidence High

Status Confirmed

CWE CWE-178: Improper Handling of Case Sensitivity; CWE-620: Unverified Password Change (analogous, e-mail); CWE-755

OWASP API API2:2023 Broken Authentication

Affected endpoints

- POST `/api/v1/auth/register`
- POST `/api/v1/auth/login`
- PATCH `/api/v1/users/me`
- PATCH `/api/v1/users/{user_id}`

Affected code

- `app/models/user.py:65`
- `app/services/auth_service.py:63`
- `app/services/auth_service.py:109`
- `app/routes/users.py:84-90`
- `app/services/user_service.py:159-170`
- `app/models/user.py:106`

Evidence / code path. `users.email` is a case-sensitive UNIQUE column and lookups compare it exactly. Pydantic `EmailStr` only lowercases the domain, so `'Sec.Case@x'` and `'sec.case@x'` become two accounts. PATCH `/users/me` accepts a new email with no current-password confirmation and no verification (`email_verified_at` is never written), and a taken address hits the DB constraint, giving an unhandled `IntegrityError` and a 500.

Preconditions. Any account. The takeover-persistence variant needs a stolen access token.

Attack scenario. Someone with a briefly hijacked access token changes the account's e-mail, which becomes a permanent pivot once a reset/recovery flow is added. Separately, a person registers the case-variant of a known applicant's address so staff see two 'same' people; squatting on addresses is possible because registration never verifies them.

Impact. Identity confusion and duplicate accounts. It weakens future account-recovery flows, and the 500s disclose the existence of addresses.

Safe reproduction / regression test. `test_fapi_sec_009_email_is_case_insensitive_unique` (FAIL: second registration 200) and `test_fapi_sec_009_self_email_change_to_taken_address_is_a_409` (FAIL: 500 `UniqueViolationError`).

Root cause. E-mail is treated as an opaque string, and identity changes are not treated as security-sensitive.

Remediation. Normalise e-mail to lower-case at every entry point and add a unique index on `lower(email)` via migration. Require `current_password` (and ideally re-verification) to change e-mail. Catch `IntegrityError` and return 409. Add e-mail verification at registration before the account can hold applications.

Minimal patch (illustrative, not applied)

```
# schemas: validator normalising EmailStr -> str.lower()
# migration: CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));
# users.py update_my_profile: if 'email' in data: require payload.current_password and verify_password(...)
```

Verification after fix. Not tested

FAPI-SEC-010 — Rate limits and audit IPs are keyed on the reverse proxy's address, not the client's

Severity

Medium

Confidence Medium

Status Needs Environment Verification

CWE CWE-348: Use of Less Trusted Source; CWE-770

OWASP API API4:2023 Unrestricted Resource Consumption; API8:2023 Security Misconfiguration

Affected endpoints

- POST `/api/v1/auth/login` (10/min)
- POST `/api/v1/auth/register` (5/min)
- POST `/api/v1/auth/refresh` (30/min)
- POST `/api/v1/auth/change-password` (5/min)
- POST `/api/v1/public/eligibility` (6/min)
- POST `/api/v1/public/apply-intents` (30/min)

Affected code

- `app/core/rate_limit.py:15`
- `app/routes/auth.py:31-32`
- `app/routes/users.py:33-34`
- `Dockerfile:43`

Evidence / code path. `slowapi` uses `get_remote_address`, i.e. `request.client.host`. The container starts `uvicorn app.main:app --host 0.0.0.0 --port $PORT` with no `--forwarded-allow-ips`, and `uvicorn 0.52` trusts `X-Forwarded-For` only from `FORWARDED_ALLOW_IPS` (default `127.0.0.1`, `uvicorn/config.py:357`), which `render.yaml` does not set. Behind Render's proxy, `request.client.host` is therefore the proxy's address for every client.

Preconditions. Deployment behind Render's (or any) reverse proxy whose address is not 127.0.0.1. Must be confirmed by logging request.client.host in production.

Attack scenario. An anonymous caller sends 10 login attempts a minute and exhausts the shared bucket, so every legitimate user gets 429 at login. Six eligibility submissions a minute shut down the public lead funnel in the same way. Activity logs record the proxy IP for every login, which defeats forensics.

Impact. A cheap denial of service against authentication and lead capture, and loss of source-IP attribution in the security audit trail. Per-attacker throttling also becomes meaningless: the limits only work globally.

Safe reproduction / regression test. Environment-dependent. Verification: in production, log request.client.host and X-Forwarded-For for a request and confirm they differ; or send 11 logins from one IP and confirm a second IP is also rate-limited.

Root cause. Proxy-aware client IP resolution is not configured.

Remediation. Run uvicorn with --proxy-headers --forwarded-allow-ips=<Render's proxy CIDR> (or '*' only if the container is unreachable except through the proxy), and keep get_remote_address. Add per-account limits on login in addition to per-IP limits.

Minimal patch (illustrative, not applied)

```
--- a/Dockerfile
+++ b/Dockerfile
-CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
+CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000} --proxy-headers --forwarded-allow-ips=${FORWARDED_ALLOW_IPS:-127.0.0.1}"]
```

Verification after fix. Not tested

FAPI-SEC-011 — Account lockout enables targeted lockout of any account and confirms which e-mails are registered

Severity **Low**

Confidence High

Status Confirmed

CWE CWE-645: Overly Restrictive Account Lockout Mechanism; CWE-204: Observable Response Discrepancy

OWASP API API2:2023 Broken Authentication

Affected endpoints

- POST /api/v1/auth/login
- POST /api/v1/auth/register

Affected code

- app/services/auth_service.py:71-72
- app/services/auth_service.py:91-101
- app/services/auth_service.py:109-111

Evidence / code path. Five wrong passwords lock any account for 15 minutes, keyed on the account alone. A locked account answers 403 'Account temporarily locked', while an unknown address always answers 400 'Invalid email or password', so the lock message confirms that the address exists. The code equalises bcrypt timing for this purpose, but the lock message undoes it. Registration also returns 409 for existing addresses.

Preconditions. Knowledge of a target e-mail (for example staff addresses on the website).

Attack scenario. An attacker sends 5 bad passwords for every counsellor's address every 15 minutes, keeping the whole console locked out. The 403-vs-400 difference enumerates staff accounts.

Impact. Denial of service against specific users and e-mail enumeration.

Safe reproduction / regression test. `test_fapi_sec_011_locked_account_is_indistinguishable_from_unknown` (FAIL: locked=403 'Account temporarily locked...' vs unknown=400 'Invalid email or password').

Root cause. Lockout keyed on the account alone, with a distinguishable response.

Remediation. Return the same status and message for locked, wrong-password and unknown-account cases; notify the account owner out of band. Prefer progressive delays or a CAPTCHA keyed on account plus IP over a hard lock. Accept 409 on register as a product trade-off or move to verify-by-email signup.

Minimal patch (illustrative, not applied)

```
-         raise ForbiddenException("Account temporarily locked due to failed login attempts")
+         verify_password(password, _dummy_hash())
+         return None # indistinguishable from a bad password; alert the owner out of band
```

Verification after fix. Not tested

FAPI-SEC-012 — Uploads are read fully into memory before the size check, with no request-body cap

Severity

Low

Confidence Medium

Status Needs Environment Verification

CWE CWE-400: Uncontrolled Resource Consumption

OWASP API API4:2023 Unrestricted Resource Consumption

Affected endpoints

- POST /api/v1/documents/upload
- POST /api/v1/student/me/documents/{document_id}/replace
- POST /api/v1/communication/threads/{thread_id}/messages (up to 10 files + voice)
- POST /api/v1/users/me/avatar
- POST /api/v1/users/{user_id}/avatar
- POST /api/v1/leave-requests
- POST /api/v1/imports/catalogue (non-production)

Affected code

- `app/core/uploads.py:116-120`
- `app/routes/communication.py:59`
- `app/routes/imports.py:62`

Evidence / code path. `store_upload` does `content = await file.read()` and only then compares `len(content)` with `MAX_UPLOAD_SIZE_MB` (25). No middleware or server limit caps the request body, and a single message may carry 10 attachments plus a voice note, all held in memory at once.

Preconditions. Any authenticated user (students included).

Attack scenario. A few concurrent multipart requests carrying 10 x 200 MB parts each push the 'starter' Render instance into memory exhaustion before any size check runs.

Impact. Availability. The actual ceiling depends on Render's proxy body limit, which was not verified.

Safe reproduction / regression test. Not executed (would be a DoS test). Code-level evidence only; verify Render's max request size.

Root cause. Size is validated after buffering rather than while streaming.

Remediation. Reject on Content-Length over the limit early (middleware); read UploadFile in chunks and abort once the limit is exceeded; set a proxy or server body cap; lower MAX_ATTACHMENTS or cap the total size per message.

Minimal patch (illustrative, not applied)

```
+ chunks, total = [], 0
+ while chunk := await file.read(1024 * 1024):
+     total += len(chunk)
+     if total > limit:
+         raise BadRequestException(...)
+     chunks.append(chunk)
+ content = b"".join(chunks)
```

Verification after fix. Not tested

Hardening recommendations and environment observations

FAPI-SEC-013 — Signed Cloudinary delivery URLs for private documents never expire

Severity	Informational
Confidence	High
Status	Informational
CWE	CWE-613: Insufficient Session Expiration (analogous)
OWASP API	API8:2023 Security Misconfiguration

Affected endpoints

- GET /api/v1/documents/{document_id}/link | download
- GET /api/v1/communication/attachments/{attachment_id}/link | download

Affected code

- app/core/uploads.py:154-205 (documented at :176-190)

Evidence / code path. The code documents this itself: cloudinary_url ignores expires_at for the authenticated delivery type without token auth, so a minted URL is a permanent bearer credential.

Preconditions. A signed URL leaks (history, logs, referrer, forwarding).

Attack scenario. A passport scan's signed URL pasted into a support ticket stays valid forever.

Impact. Long-lived access to private documents after a URL leaks.

Safe reproduction / regression test. Documented in code; not re-tested.

Root cause. Cloudinary token-based authentication is not enabled.

Remediation. Enable Cloudinary token-based auth (auth_token with a short duration) for the authenticated type, or proxy file bytes through the API with Cache-Control: private, no-store.

Minimal patch (illustrative, not applied)

n/a (account configuration + build_download_url auth_token)

Verification after fix. Not tested

FAPI-SEC-014 — Counsellors can read and edit any student's profile and documents regardless of assignment

Severity

Informational

Confidence High

Status Informational

CWE CWE-285: Improper Authorization (policy gap)

OWASP API API1:2023 Broken Object Level Authorization

Affected endpoints

- /api/v1/users/{user_id}/student-profile|education|experience|research|shortlist
- /api/v1/documents* (staff branch)
- PATCH /api/v1/users/{user_id} (counsellor on students)

Affected code

- app/routes/student_profile.py:26-39
- app/routes/document.py:71-73
- app/routes/users.py:212-218

Evidence / code path. The documented own-work rule (api/scoping.py) covers leads, applications and threads only. Profiles (passport and citizenship numbers, family and addresses) and documents are open to every counsellor.

Preconditions. Counsellor account.

Attack scenario. A counsellor browses passport numbers of students assigned to colleagues.

Impact. Over-broad internal access to PII. This is a policy decision, not a code defect, but it is inconsistent with FAPI-SEC-005's rule.

Safe reproduction / regression test. Code review.

Root cause. Scoping policy is defined per surface.

Remediation. Decide the policy explicitly; if own-work applies to people, apply may_see_record-style checks (student's assigned counsellor) in student_profile and document staff branches.

Minimal patch (illustrative, not applied)

n/a (policy decision)

Verification after fix. Not tested

FAPI-SEC-015 — Managers and admins can approve their own leave requests

Severity

Informational

Confidence High
Status Informational
CWE CWE-841: Improper Enforcement of Behavioral Workflow
OWASP API API6:2023 Unrestricted Access to Sensitive Business Flows

Affected endpoints

- POST /api/v1/leave-requests/{request_id}/approve
- POST /api/v1/leave-requests/{request_id}/reject

Affected code

- app/routes/leave.py:205-216

Evidence / code path. approve_leave_request checks role and PENDING status but not request.user_id != user.id. The status check is not atomic, so there is also a double-approval race.

Preconditions. Manager/admin account.

Attack scenario. A manager submits and approves their own paid leave; attendance is marked as leave.

Impact. Segregation-of-duties gap in HR and payroll.

Safe reproduction / regression test. Code review.

Root cause. Missing self-approval rule.

Remediation. Refuse approval or rejection when request.user_id == user.id (unless super_admin); use UPDATE ... WHERE status='pending' to make the transition atomic.

Minimal patch (illustrative, not applied)

```
+ if request.user_id == user.id and user.role is not UserRole.SUPER_ADMIN:  
+     raise ForbiddenException("You cannot review your own leave request")
```

Verification after fix. Not tested

FAPI-SEC-016 — Access tokens survive logout and password change; refresh rotation is not atomic

Severity **Informational**
Confidence High
Status Informational
CWE CWE-613: Insufficient Session Expiration; CWE-367: TOCTOU
OWASP API API2:2023 Broken Authentication

Affected endpoints

- POST /api/v1/auth/logout
- POST /api/v1/auth/change-password
- POST /api/v1/auth/refresh

Affected code

- app/core/security.py:40
- app/services/auth_service.py:207-224

Evidence / code path. Access JWTs carry no session id and are valid for 15 minutes after the sessions are revoked. `refresh_tokens` selects the session row without `FOR UPDATE` and then revokes it, so two concurrent refreshes with the same token can both succeed.

Preconditions. A stolen access token (15-minute window), or a racing refresh.

Attack scenario. After the victim changes their password, an attacker's stolen access token keeps working for up to 15 minutes.

Impact. Short residual access; refresh-replay detection is weakened.

Safe reproduction / regression test. Code review.

Root cause. Stateless access tokens, and non-atomic rotation.

Remediation. Accept the 15-minute window or add a session id (`sid`) claim checked against `user_sessions`. Rotate with `UPDATE ... WHERE revoked_at IS NULL RETURNING (or SELECT ... FOR UPDATE)`, and treat reuse of a revoked token as compromise (revoke the whole family).

Minimal patch (illustrative, not applied)

n/a

Verification after fix. Not tested

FAPI-SEC-017 — The developer `.env` holds hosted Neon database and Cloudinary credentials under `ENVIRONMENT=development`

Severity

Informational

Confidence Medium

Status Needs Environment Verification

CWE CWE-798/CWE-260 (credential handling hygiene)

OWASP API API8:2023 Security Misconfiguration

Affected endpoints

- n/a (developer workstation)

Affected code

- `.env` (untracked; confirmed ignored by `.gitignore:2` and `.dockerignore:3`; not present in git history)

Evidence / code path. `.env` sets `DATABASE_URL` to a Neon host (`*.us-east-2.aws.neon.tech/neondb`) plus `CLOUDINARY_*` secrets, with `ENVIRONMENT=development`. Values were not printed. If this is the production database, a local run exposes production data through a development-mode server (`/docs` enabled, permissive LAN CORS regex, per-process JWT secret).

Preconditions. Developer machine compromise, or running dev tooling against production data.

Attack scenario. A local `uvicorn --reload` bound to a LAN interface serves production data to any device on the network whose origin matches the dev CORS regex.

Impact. Production data exposure through development configuration.

Safe reproduction / regression test. Inspection with values redacted.

Root cause. The same hosted database and media account appear to serve development and production.

Remediation. Confirm whether this is the production database. Use a separate Neon branch or project and a separate Cloudinary environment for development; rotate the credentials if they are production ones.

Minimal patch (illustrative, not applied)

n/a

Verification after fix. Not tested

FAPI-SEC-018 — Anonymous eligibility submissions attach to existing leads matched by e-mail or phone

Severity

Informational

Confidence Medium

Status Informational

CWE CWE-841: Improper Enforcement of Behavioral Workflow

OWASP API API6:2023 Unrestricted Access to Sensitive Business Flows

Affected endpoints

- POST /api/v1/public/eligibility

Affected code

- app/services/eligibility_service.py:120-158

Evidence / code path. submit() resolves an existing lead by e-mail or phone and attaches the new assessment to it (and fills lead.email when empty). The caller is anonymous and supplies both fields.

Preconditions. Knowledge of a lead's e-mail or phone.

Attack scenario. Someone submits fabricated answers under a real prospect's phone number; the counsellor sees them on that person's lead. A phone of lead A combined with the e-mail of lead B can hit the unique e-mail index and return 500.

Impact. CRM data integrity. No data is disclosed (the response is deliberately narrow).

Safe reproduction / regression test. Code review.

Root cause. The identity of an anonymous submitter is inferred from self-asserted contact details.

Remediation. Always create a new assessment linked as 'unverified submission', and let staff merge it; or require e-mail confirmation before attaching to an existing lead.

Minimal patch (illustrative, not applied)

n/a

Verification after fix. Not tested

9. OWASP API Security Top 10 (2023) coverage

Category	Status	Findings	Notes
API1 BOLA	Finding(s)	FAPI-SEC-002, FAPI-SEC-003, FAPI-SEC-004, FAPI-SEC-005, FAPI-SEC-014	Every id-taking route traced; portal repository and most inline checks sound; gaps are child ids and scoping on newer routers.
API2 Broken Authentication	Finding(s)	FAPI-SEC-009, FAPI-SEC-011, FAPI-SEC-016	JWT alg pinning, token type separation, refresh rotation and bcrypt verified; e-mail/lockout issues.
API3 Object Property Level	Finding(s)	FAPI-SEC-006	Mass assignment checked on every create/update schema (extra=forbid on register and self-update; role/status/verified_* not client-writable). Leave attachment exposure.
API4 Resource Consumption	Finding(s)	FAPI-SEC-007, FAPI-SEC-010, FAPI-SEC-012	Public limits clamped; staff/student lists and uploads not.
API5 Function Level Authorization	Finding(s)	FAPI-SEC-008	Build-time guard on auth markers; draft CMS reads too broad.
API6 Sensitive Business Flows	Finding(s)	FAPI-SEC-001, FAPI-SEC-002, FAPI-SEC-015, FAPI-SEC-018	Payment/paywall, checklist verification, leave approval, public lead capture.
API7 SSRF	Reviewed, no confirmed finding	—	Only outbound HTTP is the landing revalidation webhook to a config-fixed URL (5s timeout). No user-controlled URLs are fetched.
API8 Security Misconfiguration	Finding(s)	FAPI-SEC-001, FAPI-SEC-006, FAPI-SEC-010, FAPI-SEC-013, FAPI-SEC-017	Docs disabled in production, generic 500s, explicit CORS origins with credentials (no wildcard); fail-open payment flag and proxy config.
API9 Improper Inventory Management	Reviewed, no confirmed finding	—	Legacy /messages router intentionally still mounted (documented); no debug or test routes; one API version.
API10 Unsafe Consumption of APIs	Reviewed, no confirmed finding	—	Cloudinary SDK responses used only for URL and size; webhook response ignored apart from status; TLS verification on by default.

10. Dependency and security-tool results

Tool	Version	Result	Output
pytest (existing suite)	9.1.1	440 passed, 0 failed (exit 0)	tool-results/pytest.txt
ruff check .	0.9.2	6 style findings, none security-relevant (exit 1)	tool-results/ruff.txt
mypy app	1.14.1	3 typing errors; the text() one was triaged as a false positive	tool-results/mypy.txt
security regression suite	—	16 failed (reproductions), 14 passed (controls)	tool-results/security-tests-pre-fix.txt
bandit	not available	Not run	Proposed: uvx bandit -r app scripts -f json -o security-audit/tool-results/bandit.json
pip-audit	not available	Not run: dependency advisories NOT assessed	Proposed: uvx pip-audit --path .venv/lib/python3.12/site-packages -f json -o security-audit/tool-results/pip-audit.json
semgrep	not available	Not run	Proposed: uvx semgrep --config p/python --config p/fastapi --json -o security-audit/tool-results/semgrep.json app
secret scanner	not configured	Manual check: .env untracked and ignored; not in git history; values redacted	—

Installed versions of security-relevant packages are recorded in tool-results/installed-versions.txt. They are pinned exactly in pyproject.toml, but there is no lockfile covering transitive dependencies. No advisory claims are made without pip-audit output.

11. Positive security controls observed

- A structural authorization guard: a test fails the build for any route without an auth marker and for any change to the public endpoint set.
- Public registration forbids role/status with extra='forbid'. The rank-based can_manage_target stops admins from creating or promoting super_admins (verified).
- JWT: algorithm allow-list, token type separation, per-request user status check, production secret validation. Refresh tokens are server-side, hashed, single-use (verified) and revoked on password change.
- Password hashing avoids bcrypt's 72-byte truncation and NUL-byte pitfalls; login timing is equalised.
- Uploads: extension allow-list, generated storage names (client filenames never reach storage), size limit, and private documents served only through ownership-checked, signed Cloudinary URLs; the paywall is enforced where the bytes are

served.

- Stored-XSS defence: message HTML is sanitised on input with an allow-list and href scheme filtering (8 bypass payloads tested, all neutralised).
- StudentScopedRepository makes student-portal queries owner-filtered by construction; other students' ids return 404, not 403.
- The public API returns published data only, clamps limits, rate-limits both write endpoints, and never caches authenticated requests.
- Operational: generic 500 bodies with request ids, JSON logs in production, docs disabled in production, non-root container, .env excluded from git and from the Docker context, hermetic test DB with a destructive-operation name guard.

12. Remediation roadmap

Priority	Findings	Why
P0: before or at next production deploy	FAPI-SEC-001, FAPI-SEC-010 (verify, then config)	Revenue bypass on a public flow; configuration-only fixes.
P1: next sprint	FAPI-SEC-002, FAPI-SEC-005, FAPI-SEC-003, FAPI-SEC-004	Object-level authorization gaps; small, local code changes with tests already written.
P2: planned	FAPI-SEC-007, FAPI-SEC-008, FAPI-SEC-009, FAPI-SEC-011, FAPI-SEC-012, FAPI-SEC-006	Resource limits, identity hygiene, sensitive-file storage.
P3: hardening / policy	FAPI-SEC-013 to FAPI-SEC-018	Defence in depth, policy decisions, environment hygiene.

Details, dependencies and the regression test for each item are in SECURITY_REMEDIATION_PLAN.md.

13. Verification and regression-test results

```
Command: ENVIRONMENT=test .venv/bin/pytest security-audit/tests/security -q -p no:cacheprovider --rootdir=. -c /dev/null -o asyncio_mode=auto -o asyncio_default_fixture_loop_scope=session -o asyncio_default_test_loop_scope=session
```

Result on the audited commit: 16 failed (each is a reproduction of the finding named in the test), 14 passed (positive controls). Fixes were not applied, because the specification allows changing application code only on explicit request. Verification after fix is therefore 'Not tested' for every finding.

14. Residual risks and items requiring runtime verification

- Dependency advisories were not assessed (pip-audit unavailable). Run it before relying on this report for supply-chain assurance.
- FAPI-SEC-010: confirm request.client.host behind Render and Render's forwarded-header behaviour.
- FAPI-SEC-012: confirm Render's maximum request body size.
- FAPI-SEC-017: confirm whether the developer .env points at the production Neon database.

- Production environment variables in the Render dashboard (SIMULATED_PAYMENTS, CORS_ORIGINS, LANDING_*) were not visible to this audit.
- Cloudinary account settings (strict transformations, token auth, allowed delivery types) were not reviewed.
- The landing site's /api/revalidate and /api/preview handlers (which receive LANDING_REVALIDATE_SECRET) are out of scope.
- No negative authorization tests exist for most staff routes beyond the ones added here; the absence of further findings does not mean the system is secure.

Appendix A: Reviewed files

- app/main.py, app/api/* (auth, student, scoping, deps, exceptions, router)
- app/core/* (config, security, rate_limit, middleware, logging, uploads, sanitize, landing, public_cache, cache, rbac, events, subscribers)
- app/routes/*: all 29 router modules (every handler)
- app/services/*: every service reached by an object-level decision (auth, user, student_profile, document, workflow, communication, message_scope, portal_access, application, eligibility, leave, notification, apply_intent, academic)
- app/schemas/*: every create/update schema, for mass assignment
- app/models/user.py, student_portal.py, payment.py (constraints and properties)
- Dockerfile, docker-compose.yml, render.yaml, start_backend.sh, .gitignore, .dockerignore, .env.example, .env (keys only), pyproject.toml
- scripts/xlsx_reader.py, scripts/extract_xlsx.py (import parser), tests/conftest.py, tests/test_endpoint_authorization.py, tests/test_scoping.py

Appendix B: Commands run

```
git rev-parse HEAD; git ls-files; git log --all -p -S 'CLOUDINARY_API_SECRET='  
ENVIRONMENT=test .venv/bin/python <route inventory via tests.test_endpoint_authorization._iter_api_routes>  
.venv/bin/pytest -q -p no:cacheprovider  
.venv/bin/ruff check . --no-cache --output-format concise  
.venv/bin/mypy app --no-incremental  
ENVIRONMENT=test .venv/bin/pytest security-audit/tests/security -q -p no:cacheprovider ...  
python3 security-audit/build_report.py  
chromium --headless --print-to-pdf=security-audit/SECURITY_AUDIT_REPORT.pdf security-  
audit/SECURITY_AUDIT_REPORT.html
```

Appendix C: Tool and runtime versions

Component	Version
Python	3.12.3
fastapi / starlette / uvicorn	0.141.1 / 1.3.1 / 0.52.0
pydantic / SQLAlchemy / asyncpg	2.13.4 / 2.0.51 / 0.31.0
PyJWT / bcrypt / python-multipart	2.13.0 / 5.0.0 / 0.0.32
pytest / ruff / mypy	9.1.1 / 0.9.2 / 1.14.1

Appendix D: False positives and accepted observations

Source	Item	Disposition
mypy	app/services/application_service.py:102 TextClause in ColumnElement list	Not a vulnerability: text() with a bound parameter (:reference_serial) and no interpolation.
manual	app/services/academic_service.py:60 getattr(self.model, self.order_by_field)	Not exploitable: order_by_field is a server-side class attribute, not request input.
manual	LIKE searches do not escape % and _	Not injection (bound parameters); at most broader matches.
manual	scripts/xlsx_reader.py parses workbook XML with xml.etree and zipfile	Accepted: admin-only endpoint, hard-disabled when ENVIRONMENT=production (imports.py:56). Zip-bomb/entity-expansion hardening recommended if ever enabled.
manual	X-Request-ID echoed from client (middleware.py:23)	Accepted: header values cannot carry CR/LF; production logs are JSON-encoded. Recommend length/charset validation.
manual	docker-compose worker references app.core.celery_app, which does not exist	Inventory note: profile-gated and inert; no background job surface exists (events are in-process).
ruff	6 style findings (I001, SIM105, UP017, SIM103, F401)	No security relevance.